ADMIOS FOUNDRY · AGENTIC CTO DISCOVERY

Build-ready spec bundle

Session: c0918e1e-5387-4239-836f-6428681064e8

Status: final

Bundle items included: 1, 2, 6, 7, 8, 11

Generated: 2026-10-28

Problem statement

NEEDS YOUR REVIEW

Problem

Everyday salaried people in Panama City are not in financial crisis (they don't run short or miss bills), but they are disorganized, have poor money habits, and lack visibility into where their money goes. As a result they leave money on the table: they don't know how much they spend per category, how much they actually save, or how much leaks out through small everyday purchases ("gastos hormiga").

- **End user (e.g., María, 32, office administration, two kids):** has barely any time to organize herself or her finances, saves very little each month, and cannot easily see where her paycheck went. Any tool must fit into about 5 minutes a day and stay simple; a complex UI makes her quit.
- **Founder (product owner):** needs to learn whether giving users this visibility actually raises their savings rate, while promising users that nobody on the founder's side can see their financial data.

Frequency

Continuous: spending happens daily, and the lack of visibility is felt every month when the paycheck is gone with little saved. Target usage is a daily check-in of about 5 minutes.

Cost

Not quantified by the founder. The cost is foregone savings: money that could be saved without lowering quality of life. The success target implies the founder believes a typical user can raise their savings rate by more than 5 percentage points of income (e.g., from 5% to over 10%) without giving up what they enjoy.

Why now

No external driver was given (no fixed date). The founder wants to validate the idea with a 10-person free pilot of strangers on a ~$10,000 USD budget before adding advice features and a paid subscription.

Persona + permissions map

NEEDS YOUR REVIEW

Personas

María — Pilot User (end user)

- **Role / job-to-be-done:** A salaried office administrator in Panama City (32, two kids) who wants to see each month where her paycheck went so she can save more without giving up her weekend dinners out.
- **Decision authority:** user
- **Friction tolerance / patience budget:** Very low. About 5 minutes a day. A complex UI, digging through menus, or tedious data entry will make her abandon the app.
- **What they need to see:** On opening: this month's total income, total spending, total savings. One tap away: spending by category with change vs. last month; transaction list to fix categories and confirm duplicates.

Founder — Product Owner

- **Role / job-to-be-done:** Defines what done means and needs evidence that users raise their savings rate by more than 5 percentage points.
- **Decision authority:** owner
- **Friction tolerance / patience budget:** Wants a simple, lean pilot within ~$10,000; no deadline.
- **What they need to see:** Only results that individual users explicitly choose to share. No access to any user's raw financial data.

Permissions matrix

Capability / data	Pilot User	Founder / operator
View own transactions, dashboard, categories	✅	—
View another user's financial data	❌	❌
Add / import / edit own transactions	✅	❌
Recategorize & create "always" store rules (own)	✅	❌
Confirm / reject duplicate removal	✅	❌
Opt in / out of sharing results	✅	❌
View shared results of opted-in users	❌	✅ (only what the user shares)
Delete account and all data	✅ (own)	❌
Edit standard category list	❌	✅ (review/approve proposed set)

> Tension: the owner needs evidence the pilot works (control need) while promising zero access to user data. Resolved by per-user opt-in sharing; exactly what a shared result contains is unresolved and goes to the reconciliation map. Operational support without data access (e.g., debugging a failed import) is also unresolved.

NFR targets

NEEDS YOUR REVIEW

Targets

Dimension	Target	Set by (role)
Concurrency	10 pilot users in year one, one bank each, ~3 months of history plus ongoing transactions. Growth target beyond the pilot not stated.	Founder (owner)
Latency	Dashboard (income, spending, savings) visible immediately on open; full daily check-in fits in ~5 minutes. Specific numeric latency not set (proposed: dashboard < 2 s on iPhone).	Pilot user
Availability	No SLA stated. Pilot-grade; an afternoon outage is an annoyance, not a business failure (assumption).	Founder (owner)
Accuracy / integrity	No duplicate counting (user confirms before removal); savings counted once even when moved to savings and also left over; categories correctable. Tolerance for small errors (e.g., a purchase in the wrong month) was asked but not answered.	Founder (owner)
Privacy / security	Only the data owner can see their financial data; no operator access; full deletion when a user leaves; sharing of results is per-user opt-in. Bank credentials/statement data must be protected (founder raised concern).	Founder (owner)
Platform / locale	iPhone only, English only, USD, Panama banks (Banco General, Banistmo, BAC, Global Bank).	Founder (owner)
Cost	Build within ~$10,000 USD; run costs kept minimal.	Founder (owner)

Named tensions

- **Speed vs. accuracy:** María needs a 5-minute check-in with no fuss; the owner needs numbers accurate enough to measure a >5pp savings change. Auto-categorization, remembered store rules, and income detection from salary deposits keep entry fast; duplicate and category prompts must be few and quick.
- **Privacy vs. measurement:** the owner wants to know if it works; users are promised nobody sees their data. Resolved via opt-in result sharing; contents of the shared payload are open.
- **Automatic bank connection vs. budget/data risk:** founder wants automatic pulls but fears data risk and has $10k; Panamanian open-banking coverage is uncertain, so statement upload is likely the primary pilot path.
- **Founder stack preference vs. cost:** Databricks + AWS was requested; at 10 users it may raise build and run costs relative to a lean backend.

Architecture recommendation

NEEDS YOUR REVIEW

Recommended approach

A native iPhone app talking to a small AWS-hosted API. The API owns users, transactions, categories, store rules, and monthly aggregates in a relational database. Statement files (Excel/CSV) are uploaded to private S3 and parsed by a per-bank parser into normalized transactions; a categorizer and duplicate detector run on ingest. Manual entry writes through the same pipeline. Dashboard and monthly category comparisons are computed server-side. Databricks is accommodated as an optional downstream analytics sink fed only with data from users who opted in to share results; it is not on the critical path of the app. Direct bank connection is a stretch item pending an aggregator feasibility check for Panama.

Stack (decided with the supervising engineer)

- **Frontend:** SwiftUI native iOS app — iPhone-only, simplest path to a fast, clean dashboard UI.
- **Backend:** Python (FastAPI) on AWS Lambda or a single small container — Python suits statement parsing (pandas/openpyxl) and lines up with the Databricks ecosystem. Pending engineer confirmation.
- **Data store:** Amazon RDS PostgreSQL (or Aurora Serverless v2) with encryption at rest; S3 for uploaded statements, deleted after parsing.
- **Hosting:** AWS (founder requested Amazon services), single region, Cognito for auth.

Key decisions made (and why)

- **Multi-tenancy:** Single shared database, strict per-user row scoping; every query filtered by authenticated user id. No shared/family accounts.
- **Auth model:** Amazon Cognito with Sign in with Apple and email; per-user data isolation enforced in the API.
- **No operator access to financial data:** no admin UI over user data; production DB access restricted and audited; field-level encryption for transaction descriptions/amounts considered.
- **Hard delete:** account deletion removes all rows, files, and any shared results.
- **No notifications, no payments, no money movement:** keeps scope and risk low.
- **Ingestion primary path = statement upload**, with manual entry; bank connection deferred behind feasibility check.
- **Rule-based categorization first** (merchant keyword map + user store rules), not ML, to fit budget.

Agentic components (where applicable)

- **Statement parsing component:** maps each bank's export format to normalized transactions (deterministic per-bank parsers; an LLM-assisted fallback is optional and must not send data to third parties without review).
- **Categorizer:** assigns standard categories using user rules first, then merchant keyword map.
- **Duplicate detector:** flags likely duplicates (same amount, close date, similar merchant) for user confirmation; never auto-deletes.

- **Income/savings detector:** identifies salary deposits and transfers to savings, computing savings without double-counting.

Open architectural questions

- Is a direct bank aggregator available for Banco General, Banistmo, BAC, Global Bank within budget? If not, upload-only for pilot.
- What do the founder's existing Databricks integrations do, and what exactly should flow into Databricks (opt-in shared results only)?
- Backend language final call (Python assumed).
- Which file formats each bank exports (XLSX, CSV, PDF?) — PDF would change parsing effort.
- Whether to use field-level encryption beyond encryption at rest.

Epic / story breakdown

NEEDS YOUR REVIEW

An iPhone app that shows salaried people in Panama City exactly where their money goes each month, so they can save more without lowering their quality of life. Users bring in one bank's transactions by uploading statements (or typing purchases), and see a simple dashboard of monthly income, spending, and savings plus a category breakdown compared with last month. The first version is a free, English-only pilot for 10 users; success is a savings-rate gain of more than 5 percentage points.

Assumptions

- Statement upload will be the main ingestion path for the pilot because direct bank connections in Panama may not fit the budget.
- Income is derived from salary deposits detected in the user's statement, and the user can correct it.
- The interviewer will propose a standard category set that the founder reviews before launch.
- All four target banks let customers export statements as spreadsheet files (XLSX/CSV).
- All amounts are in USD.
- Months are calendar months.
- A transfer to a savings account is identifiable from statement data or by user tagging.
- Users take responsibility for uploading statements periodically; no automatic sync exists without bank connection.
- Databricks is not on the critical path; it receives only opt-in shared results.

Open Questions

1. Is a direct bank connection with Banco General, Banistmo, BAC, and Global Bank feasible within the $10k budget? *(blocks: Bank Connection epic)*
2. What statement formats does each bank export, and can we get sample files? *(blocks: Upload Bank Statement story)*
3. How accurate must numbers be before users lose trust (e.g., a purchase landing in the wrong month)? *(blocks: Duplicate Detection and Monthly Dashboard stories)*
4. What exactly is included in a user's shared results (savings-rate change only, or category totals)? *(blocks: Share Results With Founder story)*
5. What do the founder's existing Databricks integrations do, and what should the app send there? *(blocks: Share Results With Founder story; architecture)*
6. Founder approval of the proposed standard category list. *(blocks: Automatic Categorization story)*
7. How does the app tell a savings transfer apart from other transfers (auto-detect, user tag, or both)? *(blocks: Compute Monthly Savings story)*
8. How are users without a detectable salary deposit handled? *(blocks: Detect Income story)*
9. How will the 10 pilot strangers be recruited and onboarded (TestFlight invites)? *(blocks: Sign Up and Sign In story)*

User Roles / Personas

María (Pilot User)

- **Role:** Office administrator in Panama City, 32, two kids, stable salary.
- **Goal:** See each month where her paycheck went so she can save more without giving up weekend dinners out.
- **Pain:** Barely has time to organize herself or her finances; saves very little and has no visibility into her spending.
- **Tech comfort:** moderate
- **Decision authority:** user

Founder (Product Owner)

- **Role:** Owner of the app and the pilot.
- **Goal:** Learn whether users raise their savings rate by more than 5 percentage points.
- **Pain:** Cannot see user data by design; depends on users choosing to share.
- **Tech comfort:** moderate
- **Decision authority:** buyer

Epics

Accounts and Privacy

Goal

Let a pilot user create a private account tied to one bank and delete it completely.

Business Value

María trusts the app with her finances because only she can see her data and she can erase it; the founder can keep the privacy promise.

Stories

Sign Up and Sign In

- **User story:** As María, I want to create an account and sign in on my iPhone, so that my financial data is private to me.
- **Acceptance criteria:**
  - Given a new user, when she signs up with Sign in with Apple or email, then an account is created and she lands on bank selection.
  - Given a signed-in user, when any API call is made, then only data owned by her user id is returned.
  - Given an unauthenticated request, when it hits any data endpoint, then it returns 401.
- **Edge cases:**
  - Email already registered: show sign-in option.
  - Auth service unavailable: show a retry message, no partial account.
- **Non-functional requirements:**

    - All traffic over TLS; tokens stored in iOS Keychain.
- **Dependencies:**
    - Amazon Cognito setup.
- **Risks:**
    - Sign in with Apple configuration effort.
- **Suggested story size:** M — auth plus per-user scoping across the API.
- **AI coding handoff notes:**
    - Context: iPhone-only pilot for 10 users; privacy is a core promise.
    - Data model: users(id, cognito_sub, email, bank, created_at, share_results bool).
    - API / integration assumptions: Cognito JWT validated in API middleware; user_id derived from token, never from request body.
    - Design boundaries: no shared/family accounts, no admin data views, no payments.
    - Test expectations: tests proving user A cannot read user B data on every endpoint.

Choose My Bank

- **User story:** As María, I want to pick my one bank during setup, so that the app reads my statements correctly.
- **Acceptance criteria:**
    - Given onboarding, when she sees bank options, then exactly Banco General, Banistmo, BAC, and Global Bank are listed.
    - Given a selected bank, when saved, then it is stored on her profile and used for statement parsing.
- **Edge cases:**
    - User's bank not in list: explain the pilot supports only these four.
- **Non-functional requirements:**
    - Onboarding step completes in under 30 seconds.
- **Dependencies:**
    - Sign Up and Sign In.
- **Risks:**
    - Users with multiple banks may feel the picture is incomplete.
- **Suggested story size:** XS — single selection screen.
- **AI coding handoff notes:**
    - Context: one bank per user for the pilot.
    - Data model: users.bank enum(BANCO_GENERAL, BANISTMO, BAC, GLOBAL_BANK).
    - API / integration assumptions: PATCH /me {bank}.
    - Design boundaries: no multi-bank support.
    - Test expectations: invalid bank values rejected.

Delete My Account Completely

- **User story:** As María, I want to delete my account, so that all my financial information is gone.
- **Acceptance criteria:**
    - Given a signed-in user, when she confirms deletion, then all transactions, rules, uploaded files, shared results, and the user record are hard-deleted.
    - Given deletion completed, when she tries to sign in, then no prior data exists.
- **Edge cases:**
    - Deletion fails partway: retry until all data is gone; user sees an error, not a false success.
    - Shared results already sent to Databricks: also deleted.

- **Non-functional requirements:**
  - Deletion completes within minutes; no soft-delete retention.
- **Dependencies:**
  - Sign Up and Sign In; Share Results With Founder (for downstream deletion).
- **Risks:**
  - Backups may retain data; backup retention policy must be short and documented.
- **Suggested story size:** S — cascade delete plus S3 and Cognito cleanup.
- **AI coding handoff notes:**
  - Context: founder requires complete deletion when a user leaves.
  - Data model: ON DELETE CASCADE from users to all owned tables.
  - API / integration assumptions: DELETE /me; remove Cognito user and S3 objects.
  - Design boundaries: no account recovery after deletion.
  - Test expectations: after delete, no rows or objects remain for that user id.

Getting Transactions In

Goal

Load three months of history and ongoing transactions from statement files or manual entry.

Business Value

Without data there is no picture; María gets month-over-month comparisons on day one.

Stories

Upload Bank Statement

- **User story:** As María, I want to upload my bank statement file, so that my transactions appear without typing them.
- **Acceptance criteria:**
  - Given a valid statement from her selected bank, when uploaded, then transactions (date, description, amount, debit/credit) are imported and shown.
  - Given onboarding, when she uploads, then she is prompted to cover the last 3 months.
  - Given parsing completes, then the raw file is deleted from storage.
- **Edge cases:**
  - Wrong bank or unsupported format: clear error, nothing imported.
  - Overlapping period with earlier upload: overlapping rows go through duplicate detection.
  - Empty or corrupted file: error message.
- **Non-functional requirements:**
  - A 3-month statement imports in under 30 seconds.
  - Files encrypted at rest and in transit.
- **Dependencies:**
  - Choose My Bank; sample statements from each bank (Open Question 2).
- **Risks:**
  - Four different formats; banks may only offer PDF.
- **Suggested story size:** L — four parsers; consider splitting into one story per bank.

- **AI coding handoff notes:**
  - Context: primary ingestion path for the pilot.
  - Data model: transactions(id, user_id, date, description, merchant_key, amount, direction, category_id, source[upload|manual], is_savings_transfer, created_at).
  - API / integration assumptions: POST /statements multipart; parser selected by user bank.
  - Design boundaries: no photos, no cash, no PDF unless confirmed.
  - Test expectations: fixture files per bank produce expected normalized rows.

Add a Purchase Manually

- **User story:** As María, I want to type in a purchase, so that I can record something not yet in a statement.
- **Acceptance criteria:**
  - Given the add form, when she enters date, amount, description, and category, then the transaction is saved and totals update.
  - Given required fields missing, when she saves, then the form shows which are missing.
- **Edge cases:**
  - Negative or zero amount: rejected.
  - Future date: rejected.
- **Non-functional requirements:**
  - Entry possible in under 20 seconds.
- **Dependencies:**
  - Sign Up and Sign In.
- **Risks:**
  - Manual entries later duplicated by uploads.
- **Suggested story size:** S — single form and endpoint.
- **AI coding handoff notes:**
  - Context: fallback ingestion path.
  - Data model: transactions with source=manual.
  - API / integration assumptions: POST /transactions.
  - Design boundaries: bank and card transactions only; no cash tracking feature.
  - Test expectations: validation tests; totals recalculated.

Confirm Possible Duplicates

- **User story:** As María, I want the app to ask me about possible duplicate purchases, so that nothing is counted twice and nothing disappears without my say.
- **Acceptance criteria:**
  - Given a new transaction with same amount, date within 3 days, and similar description as an existing one, then it is flagged as a possible duplicate.
  - Given a flagged pair, when she confirms duplicate, then one copy is removed; when she rejects, both stay and are not flagged again.
  - Flagged items are never auto-removed.
- **Edge cases:**
  - Two genuine identical purchases on the same day: user rejects, both kept.
  - Many flags after a large upload: shown as a list she can work through.
- **Non-functional requirements:**
  - Unresolved duplicates are counted once in totals until decided (assumption pending Open Question 3).

- **Dependencies:**
  - Upload Bank Statement; Add a Purchase Manually.
- **Risks:**
  - Too many false positives add friction.
- **Suggested story size:** M — matching logic plus review UI.
- **AI coding handoff notes:**
  - Context: founder asked the app to be smart about duplicates but always ask.
  - Data model: duplicate_flags(id, user_id, txn_a, txn_b, status[pending|confirmed|rejected]).
  - API / integration assumptions: GET /duplicates, POST /duplicates/{id}/resolve.
  - Design boundaries: no auto-deletion.
  - Test expectations: matching unit tests for true/false positives.

Bank Connection Feasibility Spike

- **User story:** As the Founder, I want to know whether direct bank connection is possible for the four banks within budget, so that I can decide whether to include it.
- **Acceptance criteria:**
  - A written finding per bank: available aggregator/API, cost, security model.
  - A go/no-go recommendation for the pilot.
- **Edge cases:**
  - No aggregator covers any bank: recommend upload-only.
- **Non-functional requirements:**
  - Timeboxed to 2 days.
- **Dependencies:**
  - None.
- **Risks:**
  - Limited open banking in Panama.
- **Suggested story size:** S — research timebox.
- **AI coding handoff notes:**
  - Context: founder wants automatic pulls but fears data risk.
  - Data model: none.
  - API / integration assumptions: none assumed.
  - Design boundaries: no production integration in this story.
  - Test expectations: none; deliverable is a document.

Understanding My Money

Goal

Categorize transactions, compute income and savings, and show the dashboard and category breakdown.

Business Value

Delivers the core value for María: seeing where her paycheck went, in under 5 minutes a day.

Stories

Automatic Categorization

- **User story:** As María, I want my purchases sorted into categories automatically, so that I don't have to sort them.
- **Acceptance criteria:**
  - Given an imported transaction, then a category from the standard set is assigned, using her store rules first, then the merchant keyword map, else "Other".
  - The standard set is the founder-approved list.
- **Edge cases:**
  - Unrecognized merchant: "Other".
- **Non-functional requirements:**
  - Categorization adds negligible time to import.
- **Dependencies:**
  - Upload Bank Statement; Open Question 6.
- **Risks:**
  - Cryptic Panamanian statement descriptions reduce accuracy.
- **Suggested story size:** M — keyword map plus rule engine.
- **AI coding handoff notes:**
  - Context: standard categories only for pilot.
  - Data model: categories(id, name); merchant_keywords(keyword, category_id).
  - API / integration assumptions: runs server-side on ingest.
  - Design boundaries: no custom categories, no ML.
  - Test expectations: sample descriptions map to expected categories.

Fix a Category and Remember It

- **User story:** As María, I want to fix a wrong category and optionally have the app always do that for that store, so that I correct mistakes once.
- **Acceptance criteria:**
  - Given a transaction, when she changes its category, then the app asks "Always put purchases from this store under X?"
  - If yes, a store rule is saved and applied to future and matching existing transactions; if no, only that transaction changes.
- **Edge cases:**
  - Store rule conflicts with existing rule: new rule replaces old.
- **Non-functional requirements:**
  - Correction takes two taps or fewer after opening the transaction.
- **Dependencies:**
  - Automatic Categorization.
- **Risks:**
  - Merchant name normalization inconsistent.
- **Suggested story size:** S — rule table and prompt.
- **AI coding handoff notes:**
  - Context: founder specified ask-then-always.
  - Data model: store_rules(id, user_id, merchant_key, category_id).
  - API / integration assumptions: PATCH /transactions/{id} {category_id, always:bool}.
  - Design boundaries: rules are per user only.
  - Test expectations: rule applies to new imports; no-rule path changes one row.

Detect Income

- **User story:** As María, I want the app to work out my income from salary deposits, so that savings can be measured without setup.
- **Acceptance criteria:**
  - Given imported credits, then recurring salary-like deposits are marked as income for the month.
  - She can correct the monthly income figure or mark/unmark deposits as income.
- **Edge cases:**
  - No salary detected: prompt her to enter monthly income.
  - Two salary deposits in a month (quincena): both counted.
- **Non-functional requirements:**
  - Corrections reflect immediately in the dashboard.
- **Dependencies:**
  - Upload Bank Statement.
- **Risks:**
  - Refunds or transfers misread as income.
- **Suggested story size:** M — heuristics plus correction UI.
- **AI coding handoff notes:**
  - Context: stable-salary users only; assumption approved by founder.
  - Data model: transactions.is_income bool; monthly_income_override(user_id, month, amount).
  - API / integration assumptions: server-side heuristic.
  - Design boundaries: no variable-income support.
  - Test expectations: fixtures with biweekly pay detected correctly.

Compute Monthly Savings

- **User story:** As María, I want to see how much I saved each month, so that I know if I'm improving.
- **Acceptance criteria:**
  - Savings = transfers to savings + money left over (income - spending excluding savings transfers), counted once.
  - Savings rate = savings / income, shown as a percentage.
- **Edge cases:**
  - Spending exceeds income: savings shown as negative.
  - Transfer not recognized as savings: user can mark it.
- **Non-functional requirements:**
  - Calculation reproducible and unit-tested.
- **Dependencies:**
  - Detect Income; Open Question 7.
- **Risks:**
  - Double counting if transfers are also counted as spending.
- **Suggested story size:** S — formula plus transfer flag.
- **AI coding handoff notes:**
  - Context: success metric is >5pp savings-rate increase.
  - Data model: transactions.is_savings_transfer bool.
  - API / integration assumptions: computed in monthly summary endpoint.
  - Design boundaries: no investment tracking.
  - Test expectations: no-double-count test cases.

Monthly Dashboard

- **User story:** As María, I want to open the app and immediately see this month's income, spending, and savings, so that I get the picture in seconds.
- **Acceptance criteria:**
  - Given app open, then the first screen shows total income, total spending, total savings for the current month.
  - She can switch to previous months within available history.
- **Edge cases:**
  - No data yet: prompt to upload a statement.
  - Offline: show last loaded figures with a stale indicator.
- **Non-functional requirements:**
  - Visible within 2 seconds; simple, uncluttered UI; no notifications.
- **Dependencies:**
  - Compute Monthly Savings.
- **Risks:**
  - Stale data if user forgets to upload.
- **Suggested story size:** M — main screen and summary API.
- **AI coding handoff notes:**
  - Context: founder said complex UI is the failure mode.
  - Data model: read-only from aggregates.
  - API / integration assumptions: GET /summary?month=YYYY-MM.
  - Design boundaries: no rules-of-thumb comparison, no gastos hormiga, no push notifications.
  - Test expectations: summary totals match fixture data.

Category Breakdown With Change From Last Month

- **User story:** As María, I want to see spending by category and how each changed from last month, so that I notice my habits shifting.
- **Acceptance criteria:**
  - One tap from the dashboard shows each category's total for the month, sorted by amount.
  - Each category shows the dollar change vs. the previous month.
  - Tapping a category lists its transactions.
- **Edge cases:**
  - No previous month data: change shown as "—".
  - Category new this month: change equals full amount.
- **Non-functional requirements:**
  - Loads within 2 seconds.
- **Dependencies:**
  - Automatic Categorization; Monthly Dashboard.
- **Risks:**
  - Incomplete month makes comparisons misleading.
- **Suggested story size:** M — breakdown view plus comparison.
- **AI coding handoff notes:**
  - Context: founder asked to add the change.
  - Data model: aggregate by user, month, category.
  - API / integration assumptions: GET /breakdown?month=YYYY-MM.
  - Design boundaries: no advice or budgets.

- Test expectations: change computed correctly across month boundaries.

Pilot Measurement

Goal

Let each user choose to share results with the founder.

Business Value

The founder learns whether savings rates rise >5pp without breaking the privacy promise.

Stories

Share Results With Founder

- **User story:** As María, I want to choose whether to share my results, so that I help the pilot only if I want to.
- **Acceptance criteria:**
  - Sharing is off by default; she can turn it on or off in settings.
  - When on, only the agreed result fields (e.g., monthly savings rate) are sent to the founder's analytics store; when off, nothing is sent and prior shared data is removed.
- **Edge cases:**
  - Analytics store unavailable: queue and retry, never block the app.
- **Non-functional requirements:**
  - Shared data contains no transaction details.
- **Dependencies:**
  - Compute Monthly Savings; Open Questions 4 and 5.
- **Risks:**
  - Databricks integration scope unknown.
- **Suggested story size:** M — consent toggle plus export.
- **AI coding handoff notes:**
  - Context: founder chose per-user opt-in.
  - Data model: users.share_results; shared_results(user_pseudonym, month, savings_rate).
  - API / integration assumptions: export to Databricks/S3 pending confirmation.
  - Design boundaries: no operator view of raw data.
  - Test expectations: opt-out removes shared rows; no export when off.

Confidence / judgment ledger

The map of where the supervising engineer should spend their attention.

Ledger

#	Artifact	Confidence	Basis	Needs human review?
1	Problem statement	low	reasoned live	yes
2	Persona + permissions map	low	reasoned live	yes
6	NFR targets	low	reasoned live	yes
7	Architecture recommendation	low	reasoned live	yes
8	Epic / story breakdown	low	reasoned live	yes

Items 3 (data model), 4 (integration plan), 5 (security posture), 9 (reconciliation map), and 10 (CLAUDE.md) are out of the v1 bundle (Q3).

Open questions

1. Is a direct bank connection with Banco General, Banistmo, BAC, and Global Bank feasible within the $10k budget? *(blocks: Getting Transactions In epic / Bank Connection Feasibility Spike)*
2. What statement file formats does each bank export, and can sample files be provided? *(blocks: Upload Bank Statement)*
3. How accurate must the numbers be before users lose trust (e.g., a purchase in the wrong month)? *(blocks: Confirm Possible Duplicates; Monthly Dashboard; nfr)*
4. What exactly does a user's shared result contain? *(blocks: Share Results With Founder)*
5. What do the founder's existing Databricks integrations do, and what should the app send there? *(blocks: Share Results With Founder; architecture-rec)*
6. Founder approval of the proposed standard category list. *(blocks: Automatic Categorization)*
7. How are transfers to savings identified (auto-detect, user tag, or both)? *(blocks: Compute Monthly Savings)*
8. How should users with no detectable salary deposit be handled? *(blocks: Detect Income)*
9. Final backend language/framework (Python assumed) with the supervising engineer. *(blocks: architecture-rec)*
10. How will the 10 pilot strangers be recruited and distributed the app (e.g., TestFlight)? *(blocks: Sign Up and Sign In)*

How to read this

- **High confidence** = templated from a mature domain pack; expect little change.
- **Low confidence** = reasoned live for this customer; this is where your judgment is needed.
- Supervision cost for this build: **5 of 5** artifacts are low-confidence (reasoned live — no domain pack for this vertical yet).

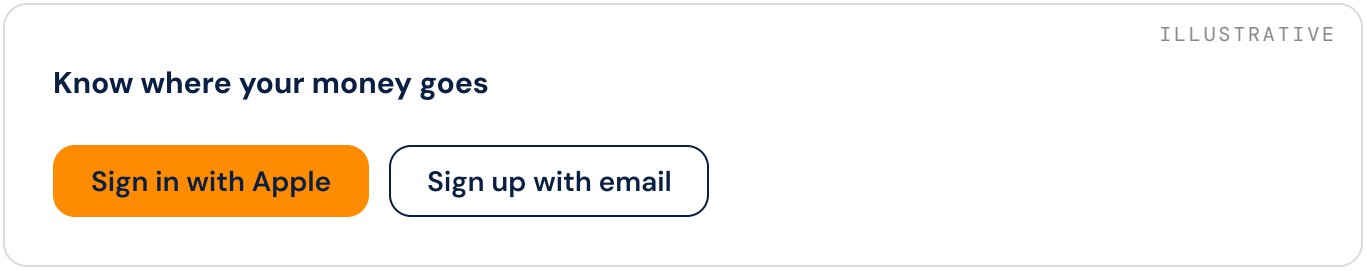
Product preview (illustrative)

An illustrative sketch of the app this spec describes, generated from the personas and stories above. Not a final design.

Welcome

María signs up or signs in with Apple or email.

Visible to: María
Covers: Sign Up and Sign In



Choose My Bank

María picks her one bank so statements are read correctly.

Visible to: María
Covers: Choose My Bank

ILLUSTRATIVE

Your bank

Bank

Banco General, Banistmo, BAC, Global Bank

Continue

The pilot supports only these four banks.

Upload Statement

María uploads the last 3 months of bank statements to load her transactions.

Visible to: María
Covers: Upload Bank Statement

ILLUSTRATIVE

Upload statement

Statement file

XLSX or CSV from your bank

Period covered

Last 3 months

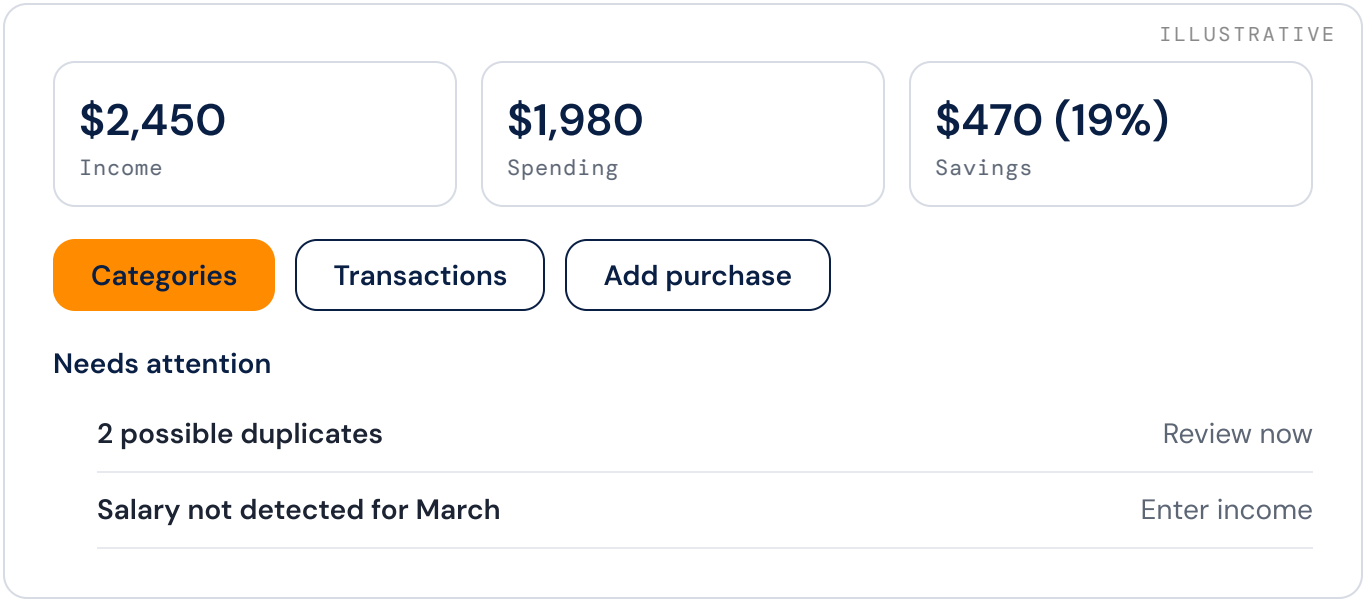
Import

Wrong bank or unsupported file: nothing is imported.

This Month

María sees this month's income, spending and savings at a glance.

Visible to: María
Covers: Monthly Dashboard; Compute Monthly Savings; Detect Income



Spending by Category

María sees each category's total and change from last month, then taps one to see its transactions.

Visible to: María
Covers: Category Breakdown With Change From Last Month

ILLUSTRATIVE

Categories, April

Category	Total	Change vs March
Groceries	$520	+$35
Dining out	$310	-$40
Transport	$180	$0
Utilities	$240	+$12
Other	$95	—

Transactions

María reviews her transactions and fixes categories.

Visible to: María
Covers: Fix a Category and Remember It; Automatic Categorization

ILLUSTRATIVE

April transactions

Date	Description	Amount	Category
Apr 3	Super Xtra Costa del Este	-$86.40	Groceries
Apr 5	Restaurante Mi Casa	-$42.00	Dining out
Apr 8	Uber trip	-$7.50	Transport
Apr 10	Salary deposit	+$1,225.00	Income
Apr 12	Cable and internet	-$58.00	Utilities

Edit Transaction

María changes a category and can make the rule permanent for that store.

Visible to: María
Covers: Fix a Category and Remember It

ILLUSTRATIVE

Restaurante Mi Casa, -$42.00

Category

Dining out

Always put this store under this category?

Is this a savings transfer?

Save

Add Purchase

María types in a purchase not yet in a statement.

Visible to: María
Covers: Add a Purchase Manually

ILLUSTRATIVE

New purchase

Date

Apr 14

Amount (USD)

0.00

Description

Store or place

Category

Choose category

Save

Possible Duplicates

María decides whether flagged purchases are duplicates; nothing is removed without her confirmation.

Visible to: María
Covers: Confirm Possible Duplicates

ILLUSTRATIVE

To review

Super Xtra, $86.40, Apr 3	Same amount, 1 day apart
Uber trip, $7.50, Apr 8	Same amount, 2 days apart

Only: María

Same purchase, remove one

Both are real, keep both

Settings

María chooses whether to share results with the founder, or deletes her account.

Visible to: María
Covers: Share Results With Founder; Delete My Account Completely

ILLUSTRATIVE

Sharing

Share my monthly savings rate with the founder

Save

Delete my account and all data